

Software Testing

Tests in Software Design

- Unit tests are essential for **iterative design**.
- They provide a **safety net** during refactoring.
- Good design makes testing **easier**.
- Tests help **verify** that our design decisions work correctly.

What Types of Tests Do We Use?

- A **unit test** checks one small behavior in isolation.
- An **integration test** checks whether multiple components work together.
- An **acceptance test** checks the system against business-facing acceptance criteria.
- A **regression test** is any test rerun after a change to detect broken existing behavior; it is a purpose, not a separate test level.

Real Example: Employee Salary Feature

The Feature: Give employees a salary raise

```
class Employee:
    def __init__(self, last_name, first_name, salary, shift=None):
        self._first_name = first_name
        self._last_name = last_name
        self.salary = salary
        self.shift = shift

    def raise_salary(self, factor):
        """Multiply current salary by the given factor"""
        self.salary = self.salary * factor

    def get_full_name(self):
        return f"{self._last_name},{self._first_name}"
```

Testing the Salary Feature

- When we add behavior, we should add focused tests that describe the expected result.
- For salary calculations, test representative normal and boundary cases.

```

import unittest

from employee import Employee

class TestEmployee(unittest.TestCase):

    def test_get_full_name(self):
        # Test basic name formatting
        employee = Employee("Doe", "John", 50000)
        self.assertEqual(employee.get_full_name(), "Doe,John")

    def test_raise_salary(self):
        # Test salary calculation
        employee = Employee("Smith", "Jane", 2000)
        employee.raise_salary(1.1) # 10% raise
        self.assertAlmostEqual(employee.salary, 2200.0)

    def test_zero_raise(self):
        # Edge case: no raise
        employee = Employee("Jones", "Bob", 3000)
        employee.raise_salary(1.0) # 0% raise
        self.assertEqual(employee.salary, 3000)

```

Unit Test

- **Unit tests** check small, observable behaviors with minimal external dependencies.
- Unit tests are **part of the development process**.
- They make refactoring safer by detecting unintended behavior changes quickly.

Arrange, Act, Assert

- **Arrange** the object and inputs.
- **Act** by calling the behavior under test.
- **Assert** the observable result.

```
employee = Employee("Smith", "Jane", 2000)      # Arrange  
employee.raise_salary(1.1)                       # Act  
self.assertAlmostEqual(employee.salary, 2200.0) # Assert
```

Running Your Tests

In Command Line:

```
python -m unittest -v test_employee.py
```

In VS Code:

- Click the green arrow next to test methods
- Right-click → "Run Tests"

Expected Output:

```
test_get_full_name ... ok  
test_raise_salary ... ok  
test_zero_raise ... ok
```

```
Ran 3 tests in 0.002s  
OK
```

Regression Testing: Preventing Backslides

Analogy: Home security system after renovation

- Test all sensors still work after changes
- Run automatically when you modify the house

In software: rerun the relevant test suite after every change. Previously passing tests then act as regression tests.

```
python -m unittest discover -v
```

- Automated CI can run the same suite on every commit or pull request.

Integration Testing: Testing Teamwork

Analogy: Testing a restaurant kitchen

- Can the chef and sous chef work together?
- Does the ordering system talk to the kitchen?

```
import io
import unittest
from contextlib import redirect_stdout
from employee import Employee
from reporting import AccountingReport

class TestEmployeeReportIntegration(unittest.TestCase):
    def test_salary_report_generation(self):
        employee = Employee("Wonder", "Alice", 5000)
        report = AccountingReport([employee])

        # Use an exact integer multiplier: this integration test checks
        # component collaboration, not floating-point tolerance.
        employee.raise_salary(2)
        output = io.StringIO()
        with redirect_stdout(output):
            report.print_report()

        result = output.getvalue()
        self.assertIn("Wonder,Alice", result)
        self.assertIn("10000", result)
```

Acceptance Testing: The Final Exam

Analogy: Test driving a completed car

- Does it start?
- Can you drive safely?
- Do all features work as advertised?

Requirements as the Guideline

- **What to Test:** Requirements describe the business outcomes the system should support.
- **How to Verify:** Derive one or more scenarios from each relevant acceptance criterion.
- **Pass/Fail Criteria:** Use clear, measurable examples agreed upon with stakeholders.

Methods:

- **Automated acceptance tests** — run business-facing scenarios automatically
- **Exploratory/manual testing** — investigate behavior that scripted checks may miss
- **User acceptance testing** — stakeholders confirm that the system meets their needs

Modern Testing with Pytest

Pytest makes testing easier and more powerful:

```
python -m pip install pytest
```

Simple test example:

```
import pytest

def calculate_bonus(salary, performance_score):
    return salary * (performance_score / 100)

def test_bonus_calculation():
    # Test normal case
    assert calculate_bonus(50000, 10) == pytest.approx(5000)

def test_zero_bonus():
    # Test edge case
    assert calculate_bonus(50000, 0) == 0

def test_high_performance():
    # Test high performer
    assert calculate_bonus(50000, 25) == pytest.approx(12500)
```

Running Pytest

```
$ pytest
===== test session starts =====
collected 3 items

test_bonus.py ... [100%]

===== 3 passed in 0.01s =====
```

If a test fails:

```
$ pytest
===== FAILURES =====
_____ test_bonus_calculation _____

    def test_bonus_calculation():
>     assert calculate_bonus(50000, 10) == pytest.approx(6000) # Wrong!
E         assert 5000.0 == 6000 ± 0.006

test_bonus.py:8: AssertionError
```

Testing Best Practices

Write Clear Test Names

Good: `test_salary_raise_with_valid_factor`
Bad: `test1`

Test Edge Cases

- Zero values, negative numbers
- Empty strings, `None` values
- Maximum/minimum boundaries

Keep Tests Fast

- Unit tests should run in milliseconds
- Inject external dependencies so unit tests can replace slow services with simple fakes or mocks.

| Keep Tests Independent

- Each test should arrange its own state and run in any order.
- Avoid shared mutable state, real networks, and reliance on another test running first.
- Test observable behavior rather than private implementation details.

Summary: Testing is Your Safety Net

Test levels:

- **Unit tests** = Testing each light switch
- **Integration tests** = Testing room connections
- **Acceptance tests** = Final home inspection

Regression purpose: After a change, rerun any of these tests to detect broken existing behavior — like checking the security system after a renovation.